

Code Explanation

Data Extraction Module Explanation

This code is a part of an ETL (Extract, Transform, Load) pipeline and is responsible for extracting data from various sources. It utilizes Apache Spark for data processing and provides a flexible way to read data from different sources with optional filtering.

Overview

The code consists of two primary functions: `read_source` and `extract_data`. The `read_source` function reads data from a single source based on the provided configuration, while the `extract_data` function orchestrates the extraction of data from multiple sources.

Step 1: Importing Necessary Modules and Defining the `read_source` Function

The code begins by importing necessary modules, including `Dict`, `List`, and `Optional` from the `typing` module for type hints, `DataFrame` and `SparkSession` from `pyspark.sql` for Spark operations, and `DataSource` from a local `.config` module, which is assumed to contain configuration details for data sources.

```
from typing import Dict, List, Optional
from pyspark.sql import DataFrame, SparkSession
from .config import DataSource
```

Step 2: Defining the `read_source` Function

The `read_source` function is defined to read data from a source configuration. It takes a `SparkSession` instance, a `DataSource` configuration, and an optional dictionary of filters.

```
def read_source(
    spark: SparkSession,
    source: DataSource,
    filters: Optional[Dict[str, str]] = None
) -> DataFrame:
```

Step 3: Initializing the Reader and Applying Options

Inside `read_source`, a reader is initialized with the specified format from the source configuration. If options are provided in the source configuration, they are applied to the reader.

```
reader = spark.read.format(source.format)
if source.options:
    for key, value in source.options.items():
        reader = reader.option(key, value)
```

Step 4: Loading Data and Applying Filters

The data is loaded from the path specified in the source configuration. If filters are provided, they are applied to the loaded `DataFrame`.

```
df = reader.load(source.path)
if filters:
    for column, value in filters.items():
        df = df.filter(f"{column} = '{value}'")
```

Step 5: Defining the extract_data Function

The `extract_data` function is defined to extract data from all configured sources. It takes a `SparkSession` instance, a list of `DataSource` configurations, and an optional date filter.

```
def extract_data(
    spark: SparkSession,
    sources: List[DataSource],
    date: Optional[str] = None
) -> Dict[str, DataFrame]:
```

Step 6: Extracting Data from Multiple Sources

Inside `extract_data`, the function iterates over the list of sources. For each source, it determines if a date filter should be applied based on the source's configuration and the provided date. It then calls `read_source` to extract the data and stores the resulting `DataFrame` in a dictionary mapped by the source name.

```
dataframes = {}
for source in sources:
    filters = {"date": date} if date and "date" in source.options.get("partitionBy", "") else None
    df = read_source(spark, source, filters)
    dataframes[source.name] = df
```

Step 7: Returning the Extracted Data

Finally, the `extract_data` function returns a dictionary containing the `DataFrames` for all the sources.

```
return dataframes
```